



CLASE 2

NÚMEROS PRIMOS

Los átomos de los números.

Taller de Programación Competitiva | 2026



PRIMO, COMPUESTO Y EL CASO DE 1

Primo

Un entero mayor o igual que 2 con exactamente dos divisores positivos: 1 y él mismo.

Compuesto

Un entero mayor o igual que 2 que puede escribirse como producto de dos enteros menores que él.

El número 1

Tiene un solo divisor positivo. Por eso no es primo ni compuesto.

DIVIDE

$d \mid n$ significa que existe un entero k tal que $n = d \cdot k$.

¿ES PRIMO?

Problema motivador

Dado un entero n , determina si sus únicos divisores positivos son 1 y n .

$$1 \leq n \leq 10^9$$

¿Qué números tendríamos que probar como posibles divisores?

n	¿Primo?
1	NO
2	SÍ
17	SÍ
25	NO
97	SÍ



PARTE 1 • PRIMALIDAD

Los divisores aparecen en pares.



UN DIVISOR TRAE UNA PAREJA

Si d divide a n , entonces n/d también divide a n .

$$n = d \cdot \frac{n}{d}$$

Por ejemplo, para $n = 36$ obtenemos los pares

$(1, 36), (2, 18), (3, 12), (4, 9), (6, 6).$

«FOREVER ALONE»

Si n es un cuadrado perfecto, $\sqrt{n}$ forma pareja consigo mismo. En 36, esa pareja es $(6, 6).$

El corte

En cada pareja, al menos uno de los factores es menor o igual que $\sqrt{n}$.

Consecuencia

Si n es compuesto, encontraremos un divisor entre 2 y $\sqrt{n}$.

PRUEBA DE PRIMALIDAD

```
1 from math import isqrt
2
3 def is_prime(n):
4     if n < 2:
5         return False
6
7     for divisor in range(2, isqrt(n) + 1):
8         if n % divisor == 0:
9             return False
10
11     return True
```

TIEMPO $\mathcal{O}(\sqrt{n})$

MEMORIA $\mathcal{O}(1)$

Detalle útil

`isqrt(n)` entrega $\lfloor \sqrt{n} \rfloor$ sin usar números decimales. Sumamos 1 porque el final de `range` no se incluye.

¿POR QUÉ LA PRUEBA ES CORRECTA?

Si encuentra un divisor

El divisor está entre 2 y $n - 1$.
Entonces n tiene más divisores que 1 y él mismo, por lo que es compuesto.

Si no encuentra ninguno

Todo número compuesto tendría un divisor menor o igual que $\sqrt{n}$. Como no existe, n es primo.



PARTE 2 • FACTORIZACIÓN

De átomos a números.



$$360 = 2 \cdot 180$$

$$180 = 2 \cdot 90$$

$$90 = 2 \cdot 45$$

$$45 = 3 \cdot 15$$

$$15 = 3 \cdot 5$$

Factorización prima

$$360 = 2^3 \cdot 3^2 \cdot 5.$$

Todo entero mayor que 1 tiene una factorización prima única.

Pensar en exponentes transforma problemas con números en problemas con pequeñas listas de cantidades.

DIVIDIR TODO LO POSIBLE

Buscamos devolver la factorización de n como pares (p, e) .

Para cada candidato p :

1. Contar cuántas veces divide al número restante.
2. Guardar el par (p, e) si aparece.
3. Continuar mientras p^2 no supere al resto.

Invariante

Antes de probar p , el resto no tiene factores primos menores que p .

Al finalizar

Si el resto es mayor que 1, es el último factor primo.

FACTORIZACIÓN POR DIVISIÓN DE PRUEBA

```
1 from math import isqrt
2 def factorize(n):
3     factors = []
4     for p in range(2, isqrt(n) + 1):
5         if p * p > n:
6             break
7         exponent = 0
8         while n % p == 0:
9             n //= p
10            exponent += 1
11        if exponent > 0:
12            factors.append((p, exponent))
13
14    if n > 1:
15        factors.append((n, 1))
16    return factors
```

Costo

Tiempo $\mathcal{O}(\sqrt{n})$ en el peor caso.

Memoria $\mathcal{O}(\log n)$ para la lista de factores.

El peor caso ocurre con un primo grande, porque no logramos reducirlo.

¿POR QUÉ EL RESTO FINAL ES PRIMO?

Supongamos que el resto $r > 1$ fuera compuesto:

$$r = a \cdot b.$$

Al menos uno de a o b sería menor o igual que $\sqrt{r}$. Ese factor tendría un primo menor o igual que $\sqrt{r}$.

Contradicción

El ciclo no habría terminado, porque todavía existiría un factor que debíamos probar.

Conclusión

El resto es 1 o es primo.

MULTIPLICAR SUMA EXPONENTES

	2	3	5	7	11
360	3	2	1	0	0
14	1	0	0	1	0
$360 \cdot 14$	4	2	1	1	0

Otra forma de mirar los números

La multiplicación suma exponentes. La división exacta los resta. Un cuadrado perfecto tiene todos sus exponentes pares.

DIVIDIR ES COMPARAR EXPONENTES

Un número d divide a n exactamente cuando, para cada primo, el exponente de d es menor o igual que el exponente de n .

	2	3	5
12	2	1	0
360	3	2	1

Uno divide al otro

$12 \mid 360$ porque cada exponente de 12 es menor o igual que el correspondiente en 360.

	2	3	5
18	1	2	0
20	2	0	1

Ninguno divide al otro

$18 \nmid 20$ por la columna de 3.
Además, $20 \nmid 18$ por la columna de 5.

¿CUÁNTOS DIVISORES TIENE 360?

NOTACIÓN

$\tau(n)$ representa la cantidad de divisores positivos de n .

Si

$$n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k},$$

cada divisor escoge independientemente un exponente entre 0 y e_i .

$$360 = 2^3 \cdot 3^2 \cdot 5^1$$

$$\tau(360) = (3 + 1)(2 + 1)(1 + 1) = 24.$$

Fórmula

$$\tau(n) = \prod_{i=1}^k (e_i + 1).$$

El número 1 corresponde a escoger todos los exponentes iguales a cero.

ENUMERAR TAMBIÉN ES COMBINAR

Para $72 = 2^3 \cdot 3^2$, elegimos una potencia de 2 y una potencia de 3.

	3^0	3^1	3^2
2^0	1	3	9
2^1	2	6	18
2^2	4	12	36
2^3	8	24	72

Límite inevitable

Listar todos los divisores cuesta al menos $\mathcal{O}(\tau(n))$, porque esa es la cantidad de valores que debemos producir.



PARTE 3 • MUCHAS CONSULTAS

Compartamos el trabajo entre todos los números.



UNA CONSULTA CAMBIÓ DE TAMAÑO

Factorizar un número hasta 10^6 por división de prueba es rápido.

Pero ahora recibimos

$$Q = 200\,000$$

números, todos entre 1 y 10^6 .

Repetimos demasiado

Probar los mismos divisores desde cero en cada consulta desperdicia trabajo.

Nueva estrategia

Preprocesar una vez todos los números hasta el máximo consultado.

CRIBA DE ERATÓSTENES

Inicialmente suponemos que todo número desde 2 es primo.

1. El primer valor no tachado es 2.
2. Tachamos sus múltiplos.
3. Repetimos con el siguiente valor no tachado.

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20

La idea

Un compuesto se descubre cuando procesamos alguno de sus factores primos.

IMPLEMENTACIÓN DE LA CRIBA

```
1 from math import isqrt
2
3 def sieve(n):
4     is_prime = [True] * (n + 1)
5     is_prime[0] = False
6     if n >= 1:
7         is_prime[1] = False
8
9     for p in range(2, isqrt(n) + 1):
10         if is_prime[p]:
11             for multiple in range(p * p, n + 1, p):
12                 is_prime[multiple] = False
13
14     return is_prime
```

¿Por qué desde p^2 ?

Los múltiplos menores, como $2p$ o $3p$, ya fueron tachados al procesar un factor menor.

TIEMPO $\mathcal{O}(n \log \log n)$

MEMORIA $\mathcal{O}(n)$

¿POR QUÉ LA CRIBA ES CORRECTA?

Todo tachado es compuesto

Solo tachamos múltiplos $p \cdot k$ con $p \geq 2$ y $k \geq 2$. Por lo tanto, cada valor tachado tiene un divisor propio.

Todo compuesto es tachado

Un compuesto tiene un factor primo no mayor que su raíz. Cuando ese primo se procesa, el compuesto aparece entre sus múltiplos.

La memoria también es una restricción

Un arreglo hasta 10^6 es razonable. Un arreglo de Python hasta 10^9 no lo es. La cota determina qué herramienta podemos usar.

PRIMERA COTA: $\mathcal{O}(n \log n)$

Al procesar un número p , recorremos como máximo n/p múltiplos. Si procesáramos todos los valores desde 2 hasta n , el trabajo sería a lo más

$$\frac{n}{2} + \frac{n}{3} + \frac{n}{4} + \cdots + \frac{n}{n} = n \left(\frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \right).$$

La suma armónica

Agrupamos denominadores entre potencias de dos. En cada bloque hay 2^k términos de valor a lo más $1/2^k$, así que el bloque suma a lo más 1. Hay $\mathcal{O}(\log n)$ bloques.

Por lo tanto, ya podemos garantizar tiempo $\mathcal{O}(n \log n)$.

LA CRIBA HACE TODAVÍA MENOS TRABAJO

El ciclo interior solo se ejecuta cuando p es primo. La suma real se parece a

$$n \left(\frac{1}{2} + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \frac{1}{11} + \cdots \right),$$

donde solo aparecen recíprocos de primos.

Una cota matemática más fina

La suma de los recíprocos de los primos hasta n crece como $\mathcal{O}(\log \log n)$. Por eso la criba toma

$$\mathcal{O}(n \log \log n)$$

tiempo.

Para usar la criba basta recordar la cota. Demostrar este resultado sobre los primos queda fuera del alcance de la clase.

GUARDEMOS UN FACTOR ÚTIL

En lugar de guardar solo si x es primo, podemos guardar

$\text{spf}[x]$ = el menor factor primo de x .

x		2	3	4	5	6	7	8	9	10
$\text{spf}[x]$		2	3	2	5	2	7	2	3	2

Factorizar

Si conocemos $\text{spf}[x]$,
dividimos por ese primo y
repetimos con el nuevo valor
hasta llegar a 1.

CRIBA DE MENOR FACTOR PRIMO

```
1 def smallest_prime_factors(n):  
2     spf = list(range(n + 1))  
3  
4     for p in range(2, n + 1):  
5         if spf[p] == p:  
6             for multiple in range(p * p, n + 1, p):  
7                 if spf[multiple] == multiple:  
8                     spf[multiple] = p  
9  
10    return spf
```

Primera marca

Solo escribimos sobre un número que todavía no recibió un factor. Así conservamos el menor.

CASO BORDE

El valor 1 tiene una factorización vacía.

FACTORIZAR UNA CONSULTA CON SPF

```
1 def factorize_with_spf(x, spf):
2     factors = []
3
4     while x > 1:
5         p = spf[x]
6         exponent = 0
7         while x % p == 0:
8             x //= p
9             exponent += 1
10        factors.append((p, exponent))
11
12    return factors
```

Cada división elimina al menos un factor 2 del tamaño posible.

POR CONSULTA $\mathcal{O}(\log x)$

PREPROCESO
 $\mathcal{O}(N \log \log N)$

¿QUÉ HERRAMIENTA CONVIENE?

Situación	Herramienta	Costo principal
Un número grande	División de prueba	$\mathcal{O}(\sqrt{n})$
Todos los primos hasta N	Criba booleana	$\mathcal{O}(N \log \log N)$
Muchas factorizaciones hasta N	Criba SPF	Preproceso y consultas

Error frecuente

Una criba no ayuda si el máximo valor es demasiado grande para guardar un arreglo. Antes de programar, estima tiempo y memoria.



PARTE 4

¿Cuántos factores primos hay dentro de un factorial?



EL EXPONENTE DE p EN $n!$

En

$$n! = 1 \cdot 2 \cdot 3 \cdots n,$$

cada múltiplo de p aporta un factor p . Los múltiplos de p^2 aportan uno adicional.

Fórmula

$$e_p(n!) = \left\lfloor \frac{n}{p} \right\rfloor + \left\lfloor \frac{n}{p^2} \right\rfloor + \left\lfloor \frac{n}{p^3} \right\rfloor + \cdots$$

Para $10!$, el exponente de 2 es $\lfloor 10/2 \rfloor + \lfloor 10/4 \rfloor + \lfloor 10/8 \rfloor = 5 + 2 + 1 = 8$.

DIVISORES DE $n!$

Con los primos $p \leq n$ y sus exponentes e_p , usamos la misma fórmula:

$$\tau(n!) = \prod_{p \leq n} (e_p + 1).$$

La criba entrega los primos y la suma de cocientes entrega cada exponente.

Desafío

No podemos construir $1000000!$, pero sí describir su factorización.

Patrón recurrente

La representación correcta evita construir un número gigantesco.

LO QUE DEBEMOS RECORDAR

Un número

Divisores en pares, primalidad y factorización en $\mathcal{O}(\sqrt{n})$.

Exponentes

Multiplicar suma exponentes. Los exponentes permiten contar y construir divisores.

Muchas consultas

La criba comparte el trabajo. SPF permite factorizar rápidamente cada valor.

La próxima clase usará estos exponentes para entender MCD, MCM e inversos.